

HYPERCLAW: Cognitive Orchestration via
Attention-Metaprotocol for Hybrid AGI Systems
Building on METTACLAW from Multi-LLM Automation (v1)
to Integrated HYPERON/PRIMUS Cognition (v2)
Preliminary Design Proposal, for Discussion Only

Ben Goertzel

February 23, 2026

Abstract

METTACLAW is a compact, operational agentic AI system implemented in METTA, featuring embedding-based long-term memory, tool use, and a transparent ~ 200 -line agent core running on PETTA. We describe HYPERCLAW, a cognitive orchestration layer that extends METTACLAW’s agentic foundations to coordinate heterogeneous AI modules—multiple LLMs, local code executors, symbolic reasoners, and neural subsystems—through an *attention-metaprotocol* governing what each module attends to and when. HYPERCLAW embeds METTACLAW as its core agent loop and adds three architectural elements: typed context frames for multi-module state management, a two-timescale attention-metaprotocol (fast-loop task execution and slow-loop strategic steering), and a module-Space abstraction wrapping heterogeneous backends behind a uniform HYPERON interface.

The architecture is proposed to be deployed in two versions.

- HYPERCLAW *v1* automates complex multi-LLM research workflows (illustrated by quantitative finance and humanoid robotics) where a human expert currently provides the creative steering that LLMs cannot yet supply on their own, using self-hosted open-source models where feasible alongside commercial APIs.¹
- HYPERCLAW *v2* deepens the same orchestration layer into the inter-module communication fabric of a full HYPERON/PRIMUS cognitive architecture, where PLN, MOSES, ECAN, WILLIAM, predictive-coding networks, METAMO motivation, and SUBREP subgoal management all interoperate over a shared ATOMSPACE.

In both regimes, the attention-metaprotocol—expressed as METTA rewrite rules over typed context frames—provides inspectable, debuggable controllability.

To be operated initially using PETTA and MORK, the stack is designed to later be compiled to a mix of PETTA and METTA-IL, and to be executed on MORK-backed ATOMSPACES with decentralized deployment on ASI:CHAIN (leveraging the latter’s rich feature-set including capability-secured Rholang processes, content-addressed provenance, and shadow/elevate/rollback lifecycle management).

Contents

1 Two Problems, One Architecture

3

¹This may potentially be leveraged within the back-end infrastructure of the “Qwestor” product, under development and to be launched by SingularityNET.

2	From MeTTaClaw to HyperClaw: Architectural Foundations	4
2.1	METTACLAW: A Minimal METTA-Native Agent	4
2.2	HYPERCLAW: Cognitive Orchestration Built on METTACLAW	5
3	The Attention-Metaprotocol in Detail	6
4	HyperClaw v1 Applications	7
4.1	Quantitative Finance Research Automation	7
4.2	Humanoid Social Robotics	8
5	HyperClaw v2: Full PRIMUS Integration	8
6	Decentralized Execution on ASI:Chain	9
A	Context Frame Schema	11
B	Attention-Metaprotocol: Formal Specification	11
B.1	Attention Directives	11
B.2	Fast-Loop Step Selection	12
B.3	Slow-Loop Mode Transitions	12
B.4	Mapping to HYPERCLAW v2 (PRIMUS) Components	12
C	Quant Finance Workflow: Detailed Architecture	13
C.1	Module Space Configuration	13
C.2	Workflow Phases as Modes	13
C.3	The Generate/Critique/Revise Pattern	14
C.4	Preventing the “Forgotten Method” Failure	14
D	Humanoid Robotics Workflow: Detailed Architecture	14
D.1	Real-Time Attention Budget	14
D.2	Conflict Resolution	15
D.3	Pedagogical Mode Transitions	15
E	ASI:Chain Deployment Architecture	16
E.1	Deployment Topology	16
E.2	Capability Contracts for Module Spaces	16
E.3	Lifecycle Management	17
F	HyperClaw v1 $\rightarrow$ v2: A Concrete Upgrade Path	17

Note on AI Usage in Doc Preparation: The appendices were LLM-generated (after extensive iterative prompting) and I read and revised but did not super-carefully scrutinize all details, they are best treated as evocative first stabs.

1 Two Problems, One Architecture

Two related but distinct challenges motivate this work.

The near-term problem: multi-agent orchestration. Modern AI work increasingly involves *multiple heterogeneous modules* cooperating on a single task: several LLMs with different strengths (increasingly self-hosted open-source models alongside commercial APIs), code execution environments, data stores, and domain-specific tools. Today this cooperation is managed by the human user, who serves as the cognitive glue—deciding which module to invoke, what context to pass, when to pivot strategy, and how to reconcile conflicting outputs. The rote orchestration is time-consuming but uncreative; the strategic steering requires domain expertise and creative judgment. An immediate need exists for a cognitive orchestration layer that automates the rote parts while keeping the human in the loop for high-level direction.

The medium-term problem: inter-process communication for AGI. The HYPERON/PRIMUS cognitive architecture assembles multiple AI methods—PLN uncertain reasoning, MOSES evolutionary program search, ECAN attention allocation, WILLIAM adaptive compression, predictive-coding neural networks, and others—into an integrated cognitive system operating over a shared ATOMSPACE metagraph. These methods need to coordinate: share context, route attention, admit or reject each other’s outputs, and transfer knowledge across tasks and paradigms. This inter-process orchestration problem is structurally identical to the multi-LLM coordination problem above, but with richer cognitive modules and formal mathematical controls (quantale weakness, geodesic effort, TRANSWEAVE transfer) governing the interactions.

The thesis of this document is that a single architectural approach—typed context frames, an attention-metaprotocol expressed as METTA rewrite rules, and a uniform module-Space abstraction—addresses both problems. In its near-term form (HYPERCLAW v1), this architecture orchestrates LLMs and code executors for practical multi-agent workflows. In its medium-term form (HYPERCLAW v2), the same architecture orchestrates PRIMUS cognitive processes for integrated AGI. The transition from v1 to v2 is incremental: the metaprotocol remains stable while the modules behind it deepen.

Two concrete examples illustrate the dual challenge.

Quantitative finance research (near-term). A researcher uses multiple LLMs with complementary strengths—one for mathematical derivations, another for creative problem-solving, a third for critical review—drawn from commercial APIs or, increasingly, self-hosted open-source models (e.g., fine-tuned LLaMA or Mixtral variants) running on local or cloud GPU infrastructure. The workflow involves roughly 100 experimental iterations across Colab notebooks, LaTeX papers, and data pipelines. The human provides two indispensable functions: (i) *strategic steering* (e.g., pivoting from explicit volatility measures to compression-based regime definitions), and (ii) *quality control* (e.g., catching when an LLM “forgets” the experimentally validated method and substitutes a hallucinated alternative when building a final library). The rote orchestration—generate/critique/revise cycles, bug-fix loops, parameter sweeps, data management—consumes most of the human’s 30 hours

but requires no creativity. In v1, HYPERCLAW automates these rote cycles. In v2, PRIMUS cognitive methods (WILLIAM pattern mining, PLN reasoning, concept blending) additionally take over much of the creative steering.

Humanoid social robotics (near-term through medium-term). A classroom education robot must simultaneously run perception (vision, speech recognition), dialogue generation, motor control, emotional appraisal, and safety monitoring. Each subsystem may use different AI backends. The orchestration challenge is real-time: which subsystem gets attention, what context each receives, and how conflicting signals (e.g., “student is engaged” from vision vs. “student is confused” from speech prosody) are resolved. In v1, HYPERCLAW manages this with LLM-based dialogue, heuristic conflict resolution, and human-designed pedagogical modes. In v2, PLN handles evidence fusion, ECAN governs attention allocation, and METAMO manages pedagogical strategy as a multi-objective motive geometry.

2 From MeTTaClaw to HyperClaw: Architectural Foundations

The architectural response to both problems begins with METTACLAW, a minimal METTA-native agent, and extends it into HYPERCLAW, a cognitive orchestration layer whose design accommodates both the near-term LLM-coordination use case and the medium-term PRIMUS-integration use case without structural discontinuity.

2.1 MeTTaClaw: A Minimal MeTTa-Native Agent

METTACLAW is a compact agentic AI system implemented entirely in METTA, running on the PETTA compiler. Its design prioritizes simplicity, transparency, and extensibility: the full agent core is approximately 200 lines of METTA code, making the control loop readily inspectable and modifiable.

The system provides three foundational capabilities:

- **Embedding-based long-term memory**, represented entirely in METTA ATOMSPACE format. The agent maintains its own memory through explicit `remember(string)` calls for storage and `query(string)` calls for retrieval of semantically related items, alongside a short-term memory of recent interactions and timestamped command traces.
- **Tool use**, including web search, file modification, communication channels, and operating-system shell access—an initial set of OPENCLAW-like capabilities.
- **Skill acquisition**, whereby the agent can learn and apply new skills and declarative knowledge through memory items, enabling incremental capability growth.

METTACLAW is operational, easy to set up, and already deployed as a lightweight agent. There is a near-term plan to integrate it into community communication platforms, where it will observe discussions, assist with coordination, summarize web content, and provide development guidance. Importantly, it is designed for collaborative development: the community can contribute code and use cases, and teach the running agent new skills via memory and tool extensions.

METTACLAW thus establishes a working foundation: a METTA-native agent loop with memory, tools, and skill learning, all transparent and extensible. What it does not yet provide is multi-module orchestration—the ability to coordinate multiple LLMs, execution environments, and cognitive subsystems on complex, multi-step tasks. Crucially, because METTACLAW is implemented in METTA on ATOMSPACE, it already lives in the same substrate that PRIMUS cognitive methods will inhabit. This means orchestration capabilities built on METTACLAW can grow from LLM coordination (v1) to full cognitive-process coordination (v2) without changing substrate.

2.2 HyperCLAW: Cognitive Orchestration Built on MeTTaCLAW

HYPERCLAW extends METTACLAW into a cognitive orchestration layer that addresses both problems identified in Section 1. It embeds the METTACLAW agent core—its memory system, tool interfaces, and skill-learning loop—and adds three architectural elements that enable coordination across heterogeneous modules, whether those modules are LLMs and executors (v1) or PRIMUS cognitive processes (v2).

1. Context frames. A *context frame* is a typed METTA structure that represents the current state of a multi-module cognitive process: active goals, working hypotheses, experimental history, module capabilities, evaluation criteria, and results so far. Context frames are ATOMSPACE objects—they can be inspected, queried, pattern-matched, and versioned. Every module invocation receives its context from (a relevant projection of) the current frame and writes its output back as a frame update. METTACLAW’s existing memory system provides the persistence layer: frame snapshots are stored via the same embedding-indexed ATOMSPACE mechanisms that METTACLAW uses for long-term memory, ensuring that project history survives across sessions.

2. The attention-metaprotocol. The attention-metaprotocol is a set of METTA rewrite rules that govern *which modules attend to what, and when*. It operates at two timescales:

- **Fast-loop (seconds–minutes):** Selects which module to invoke next, what slice of the context frame to pass, and how to integrate the response. This handles the rote orchestration: generate/critique/revise, bug-fix, parameter sweep.
- **Slow-loop (minutes–hours):** Evaluates whether the current strategic direction is working, proposes mode switches (exploration → exploitation → wrap-up), and gates creative pivots that may require human approval.

The metaprotocol is “meta” because it does not specify what individual modules compute—it specifies what they *attend to* and how their outputs are *admitted* into the shared context. The fast-loop orchestration logic is itself expressed as METTA rewrite rules, consistent with METTACLAW’s design philosophy of keeping all control flow transparent and inspectable in METTA.

3. Module Spaces. Each external module (LLM endpoint, Python executor, data store, sensor stream) is wrapped as a *HYPERON Space*—a standard interface supporting match, bind, and rewrite operations. The orchestrator interacts with all modules through the same Space API, regardless of whether the module is a cloud LLM, a local Python process, a MORK-backed knowledge graph, or a robot’s motor controller. METTACLAW’s existing tool interfaces (shell, web search, file system) become module Spaces within HYPERCLAW, and new Spaces can be added incrementally—matching METTACLAW’s emphasis on ease of extension.

Relationship summary. METTACLAW is a working METTA-native agent with memory, tools, and skill learning. HYPERCLAW is a cognitive orchestration layer that uses METTACLAW as its agent core and adds context frames, an attention-metaprotocol, and a module-Space abstraction for coordinating multiple AI systems on complex tasks. In deployment, a HYPERCLAW instance *is* a METTACLAW agent with orchestration capabilities: everything METTACLAW can do (remember, query, use tools, learn skills), HYPERCLAW can do, plus it can manage multi-module workflows. The critical design property is that none of the three added elements—context frames, attention-metaprotocol, module Spaces—assume anything about *what kind of module* sits behind a Space. In v1 the modules are LLMs and executors; in v2 they are PRIMUS cognitive processes; the orchestration layer is the same.

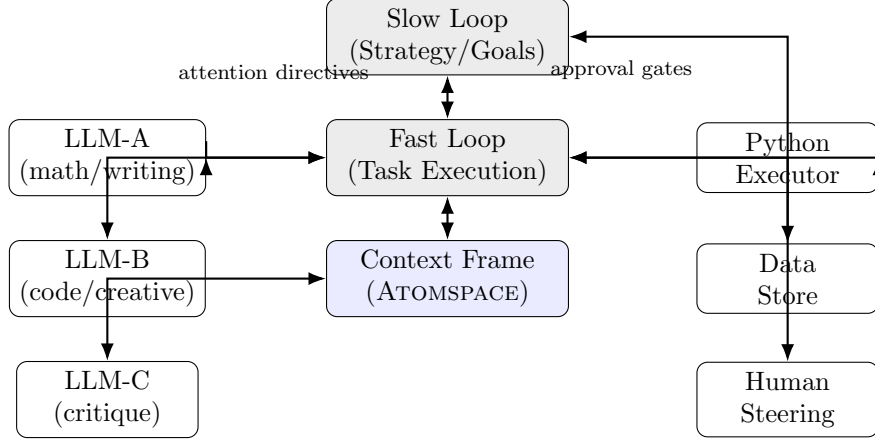


Figure 1: Top-level HYPERCLAW architecture. The slow loop governs strategy; the fast loop executes tasks; the context frame maintains shared state; module Spaces provide uniform access to heterogeneous backends. The METTACLAW agent core (memory, tools, skill learning) underlies the entire orchestration layer.

3 The Attention-Metaprotocol in Detail

The attention-metaprotocol governs inter-module communication through four mechanisms.

Attention directives. An attention directive is a typed message specifying: (i) *target module*, (ii) *context slice* (which projection of the frame to include), (iii) *task type* (generate, critique, revise, execute, evaluate), and (iv) *admission criteria* (what counts as a valid response). The fast loop emits attention directives; modules consume them and produce frame updates.

Admission gates. Not every module output gets written into the context frame. Admission gates are lightweight predicates—expressible as METTA rules—that check whether an output is consistent with the current frame. In HYPERCLAW v1, these are simple checks: does the generated code parse? Do the numerical results fall in a plausible range? Does the critique address the actual method rather than a hallucinated variant? In HYPERCLAW v2, admission gates become SUBREP certificates (CDS/PDS), providing formal guarantees that admitted options serve the declared motives.

Mode-switching logic. The slow loop maintains a *mode register* within the context frame. Modes are strategic postures—“explore methods,” “refine parameters,” “validate on held-out data,” “wrap up deliverables”—that configure the fast loop’s behavior: which modules to prefer, how much autonomy to grant, what approval gates to enforce. Mode transitions are triggered by METTA rules that evaluate accumulated evidence (e.g., “results have plateaued for N iterations”) and may require human confirmation for high-stakes pivots.

Geodesic scoring. Each candidate next-step is scored by the product $f \cdot g$, where f measures forward reachability (how feasible is this step given current resources and module capabilities?) and g measures backward usefulness (how much does this step advance the active goals?). The fast loop selects the step maximizing $\Delta(\log f + \log g)$ per unit cost. This is the same geodesic control principle used throughout PRIMUS, applied here to orchestration decisions.

4 HyperClaw v1 Applications

We now show how HYPERCLAW v1 addresses the near-term orchestration problem from Section 1, using the quant-finance and robotics examples. Section 5 then shows how the same architecture deepens into v2 for the medium-term PRIMUS integration problem.

4.1 Quantitative Finance Research Automation

The quant-finance workflow described in Section 1 maps onto HYPERCLAW as follows.

The **context frame** contains: current-best-method (regime definition, feature set, evaluation protocol), data-source registry, experimental-results history (indexed by method variant and dataset), current Sharpe ratios and transaction-cost estimates, and a deliverables checklist (LaTeX paper, Python library, example datasets).

The **fast loop** automates the rote cycles:

- *Generate/critique/revise*: Emit an attention directive to the math/writing LLM for derivation; fan out critique directives to one or more reviewer LLMs; collect responses; emit a revision directive incorporating the critiques.
- *Code/run/analyze*: Emit a code-generation directive to the preferred coding LLM; the Python executor Space runs the script locally (not in Colab—the persistent local filesystem eliminates GDrive permission friction and session-memory loss); results are captured and fed back for LLM analysis.
- *Bug-fix*: If execution fails, the error trace is automatically fed to an LLM with a “diagnose: code bug or methodology flaw?” directive.

The **slow loop** handles strategic steering:

- Evaluates whether current results justify continuing the active method or pivoting. Geodesic scoring compares the $f \cdot g$ value of “refine current approach” against “try a fundamentally different method.”
- Proposes mode switches (e.g., from “explore volatility measures” to “explore compression-based regimes”). High-impact pivots trigger a human-approval gate.
- Maintains the *certified method record*—the experimentally validated procedure that actually works. This prevents the catastrophic failure where an LLM “forgets” the real method: the context frame is the authoritative source, and the library-building step must reference certified frame entries, not LLM memory.

The **module routing table** tracks which LLM is best for which task type (math/writing, code generation, critical review), updated empirically based on output quality ratings. LLM Spaces are backend-agnostic: the same attention directive works whether the target is a self-hosted open-source model (e.g., a fine-tuned LLaMA served via vLLM), a commercial API endpoint, or a locally running smaller model for lightweight tasks. Self-hosted models are preferred where feasible, as they offer lower latency, no per-token cost at scale, full control over fine-tuning, and the ability to run on ASI:CHAIN infrastructure.

Projected impact. Based on the detailed workflow analysis, it is conjectured that HYPERCLAW automation might reduce the human’s required time from ~ 30 hours to ~ 5 hours of focused strategic input—the two creative pivots, the final review, and scattered approval decisions. A further reduction to ~ 1 -2 human-hours (skimming results and reviewing document output) may become possible in HYPERCLAW v2, when PRIMUS cognitive methods (WILLIAM pattern mining, concept blending, PLN reasoning) handle the creative interventions that v1 leaves to the human.

4.2 Humanoid Social Robotics

For a classroom robot (e.g., Mind Children’s education robot or Hanson Robotics’ expressive humanoids), HYPERCLAW orchestrates real-time cognitive processes.

The **context frame** contains: current scene graph (objects, people, spatial relations), dialogue state (conversational history, current topic, student model), emotional appraisal (valence, arousal, engagement estimates), active lesson plan, and safety constraints.

The **fast loop** runs at interactive speed ($< 500\text{ms}$ per cycle):

- Perception modules (vision, speech recognition) write observations to the frame.
- Conflict resolution rules reconcile contradictory signals (e.g., visual engagement vs. vocal confusion).
- The dialogue module receives an attention directive containing the reconciled scene/emotion state plus the active pedagogical goal.
- Motor control receives gesture directives aligned with the dialogue plan via geodesic scoring—gestures that advance both communicative clarity (g) and physical feasibility (f).

The **slow loop** operates on longer timescales ($\sim$ seconds to minutes):

- Tracks pedagogical progress against lesson objectives.
- Adjusts emotional-regulation parameters (e.g., throttling novelty when student anxiety is high—a METAMO motive-geometry adjustment).
- Triggers mode switches (“explain” $\rightarrow$ “quiz” $\rightarrow$ “encourage” $\rightarrow$ “de-escalate”).

The **attention-metaprotocol** here is time-critical: perception modules get continuous attention; dialogue and motor planning share a priority budget; safety monitoring runs as a non-preemptible background process with veto power.

5 HyperClaw v2: Full PRIMUS Integration

We now turn to the medium-term problem from Section 1: orchestrating PRIMUS cognitive processes within an integrated HYPERON AGI system.

HYPERCLAW v1, as described in the preceding sections, is deliberately simplistic: it uses LLMs as its primary cognitive modules, heuristic predicates for admission, a flat mode register for strategy, and the human for creative interventions. This is appropriate for near-term deployment and already delivers substantial automation gains.

HYPERCLAW v2 retains the same architectural skeleton—context frames, attention-metaprotocol, module Spaces—but replaces the simplistic v1 internals with the full PRIMUS cognitive toolkit. The metaprotocol does not change; what changes is the depth and sophistication of the modules it orchestrates and the formal machinery behind its control decisions.

Context frames become Atomspace regions. In v1, context frames are structured METTA objects. In v2, they become regions of the typed metagraph—the same MORK-backed ATOMSPACE that stores facts, rules, tensors, motives, and certificates. The frame-update mechanism becomes ATOMSPACE rewrite, and frame queries become pattern-matching over the metagraph.

The fast loop gains PRIMUS cognitive methods. v1’s fast loop dispatches to LLMs for generation, critique, and code. v2’s fast loop adds: PLN for uncertain reasoning (not just LLM critique), MOSES/GEO-EVO for program synthesis (not just LLM code generation), predictive-coding networks for fast forecasting, and SUBREP for certified option admission (not just heuristic checks). LLMs remain available as module Spaces—especially self-hosted open-source models for tasks where they excel—but they are now *among* the cognitive methods rather than the *only* ones.

The slow loop gains MetaMo and the ambient loop. v1’s simple mode register is replaced by METAMO’s motive geometries—a multi-objective optimization landscape that can represent complex trade-offs (performance vs. safety, exploration vs. exploitation) as explicit, auditable mathematical objects. ECAN provides the attention allocation that v1’s metaprotocol handles via hand-written rules. The ambient loop’s pattern mining (WILLIAM) and concept blending supply the creative interventions that v1 requires from the human.

Geodesic scoring deepens. The $f \cdot g$ scoring of orchestration steps, present in simplified form in v1, becomes the same Schrödinger-bridge geodesic control that governs PLN inference, MOSES evolution, and TRANSWEAVE transfer. The unified mathematical framework (quantale weakness + geodesic effort) ensures that orchestration decisions are consistent with all other cognitive decisions throughout the system.

Module Spaces expand. v1’s Spaces are primarily LLM endpoints and executors. v2 adds: PLN factor-graph Spaces for reasoning, MOSES deme Spaces for program evolution, QuantiMORK Spaces for deep neural-symbolic integration with predictive-coding waveformers, and TRANSWEAVE Spaces that manage knowledge transfer across tasks. The attention-metaprotocol routes among all of these uniformly—the same directive format that invokes an LLM in v1 invokes a PLN inference chain or a MOSES evolutionary run in v2.

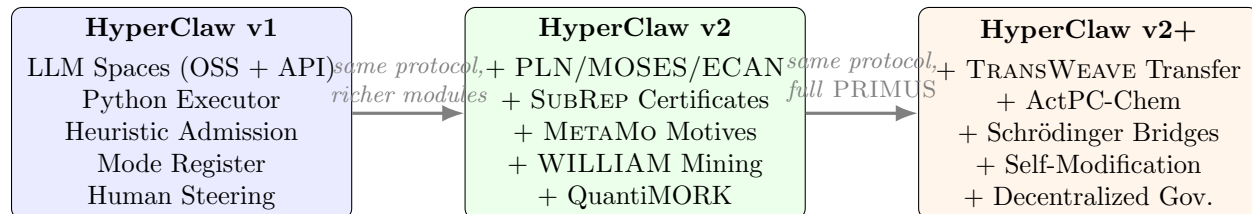


Figure 2: HYPERCLAW evolution: the attention-metaprotocol and context-frame architecture remain stable across versions; the modules behind them grow from LLM-centric (v1) to PRIMUS-integrated (v2/v2+).

6 Decentralized Execution on ASI:Chain

The entire HYPERCLAW stack is designed to be runnable within ASI:CHAIN nodes, where the METTA code is compiled to an appropriate mix of PETTA and METTA-IL and runs on capability-secured infrastructure.

Initial deployment. may occur before ASI:CHAIN is ready, but the intention is to port HYPERCLAW into ASI:CHAIN when the time is right.

Content-addressed everything. Context frames, experimental results, certified methods, LLM routing tables, and orchestration rules are all CID-addressed MORK objects with Merkle proofs. The question “what method did the system actually use?” always has a cryptographically verifiable answer.

Capability-secured execution. METTA-IL bundles compile to Rholang processes running under RSpace with least-privilege capabilities. Each module Space specifies exactly which frame regions it can read, which it can write, and what computational budget it may consume. The

Python executor, for instance, gets write access to a workspace folder and read access to specified data, but cannot modify the certified method record or the orchestration rules.

Shadow/elevate/rollback lifecycle. New orchestration strategies or module configurations deploy first in *shadow mode* (running alongside the current configuration with output comparison), graduate to *dual-run* with metrics, and *elevate* to primary only when KPIs stabilize. The last-good CID remains available for instant rollback.

Multi-party collaboration. Different teams can contribute module Spaces (e.g., a quant team contributing a specialized data-acquisition Space, a robotics lab contributing a motor-control Space) under capability contracts that ensure composability without trust assumptions. DAOs or cooperatives can gate upgrades to shared orchestration rules based on published certificates and KPIs.

Economic metering. LLM calls, compute time, and data access are metered transparently. The attention-metaprotocol’s geodesic scoring naturally minimizes cost per unit of goal-progress, creating economic pressure toward efficient orchestration.

A Context Frame Schema

A context frame $\mathcal{F}$ is a typed METTA record with the following top-level structure. All fields are ATOMSPACE objects, queryable by pattern matching.

Field	Type	Description
goals	Set(Motive)	Active goals with priority weights
mode	ModeTag	Current strategic posture
hypotheses	Map(ID, Hypothesis)	Working hypotheses with evidence
method	CertifiedMethod	Current best method (certified)
history	List(ExperimentRecord)	Chronological experiment log
results	Map(Variant, Metrics)	Results indexed by variant
modules	Map(Name, ModuleProfile)	Module capabilities and ratings
budget	ResourceBudget	Remaining compute/time/cost budget
deliverables	Checklist(Artifact)	Required output artifacts
constraints	Set(Constraint)	Safety and quality constraints

Table 1: Top-level context frame schema.

The `CertifiedMethod` type deserves special attention: it bundles a method description, the specific parameter values, the evaluation protocol, and a certificate hash linking to the experimental evidence that validated it. When the system builds a final deliverable (e.g., a clean Python library), the build process must reference the certified method’s CID—it cannot substitute content from LLM memory alone.

In HYPERCLAW v2, `goals` becomes a METAMO motive geometry, `hypotheses` carry PLN truth values (strength, confidence), and `method` certificates become SUBREP CDS/PDS admission records.

B Attention-Metaprotocol: Formal Specification

B.1 Attention Directives

An attention directive α is a 5-tuple:

$$\alpha = (\textit{target}, \textit{slice}, \textit{task}, \textit{admit}, \textit{priority})$$

where $\textit{target} \in \text{Spaces}$ identifies the module, $\textit{slice} : \mathcal{F} \rightarrow \mathcal{F}'$ is a projection extracting relevant context, $\textit{task} \in \{\text{GENERATE}, \text{CRITIQUE}, \text{REVISE}, \text{EXECUTE}, \text{EVALUATE}\}$ specifies the operation type, $\textit{admit} : \text{Output} \rightarrow \{0, 1\}$ is the admission gate, and $\textit{priority} \in \mathbb{R}^+$ is the geodesic score.

B.2 Fast-Loop Step Selection

Algorithm 1 Fast-loop step selection

Require: Context frame $\mathcal{F}$, available Spaces $\mathcal{S}$, mode m

Ensure: Updated frame $\mathcal{F}'$

```

1:  $candidates \leftarrow \text{GENERATECANDIDATEDIRECTIVES}(\mathcal{F}, \mathcal{S}, m)$ 
2: for all  $\alpha \in candidates$  do
3:    $f_\alpha \leftarrow \text{FORWARDREACHABILITY}(\alpha, \mathcal{F})$   $\triangleright$  feasibility given current resources
4:    $g_\alpha \leftarrow \text{BACKWARDUSEFULNESS}(\alpha, \mathcal{F}.goals)$   $\triangleright$  goal advancement
5:    $priority_\alpha \leftarrow \Delta(\log f_\alpha + \log g_\alpha) / \text{COST}(\alpha)$ 
6:    $\alpha^* \leftarrow \arg \max_\alpha priority_\alpha$ 
7:    $output \leftarrow \text{INVOKE}(\alpha^*.target, \alpha^*.slice(\mathcal{F}), \alpha^*.task)$ 
8:   if  $\alpha^*.admit(output)$  then
9:      $\mathcal{F}' \leftarrow \text{UPDATEFRAME}(\mathcal{F}, output, \alpha^*)$ 
10:  else
11:     $\mathcal{F}' \leftarrow \text{LOGREJECTION}(\mathcal{F}, output, \alpha^*)$ 
12: return  $\mathcal{F}'$ 

```

B.3 Slow-Loop Mode Transitions

Algorithm 2 Slow-loop mode evaluation

Require: Context frame $\mathcal{F}$, mode m , transition rules $\mathcal{R}$

Ensure: Updated mode m' , possibly with human approval

```

1:  $evidence \leftarrow \text{AGGREGATERECENTRESULTS}(\mathcal{F}.history)$ 
2: for all rule  $r \in \mathcal{R}$  do
3:   if  $r.trigger(evidence, m)$  then
4:      $m_{proposed} \leftarrow r.newMode$ 
5:      $\Delta_{fg} \leftarrow \text{GEODESICGAIN}(m_{proposed}, \mathcal{F})$   $\triangleright$  expected improvement
6:     if  $r.requiresApproval$  and  $\Delta_{fg} < \theta_{auto}$  then
7:        $m' \leftarrow \text{REQUESTHUMANAPPROVAL}(m_{proposed}, evidence)$ 
8:     else
9:        $m' \leftarrow m_{proposed}$ 
10:     $\text{UPDATEFRAME}(\mathcal{F}, mode \leftarrow m')$ 
11:    return  $m'$ 
12: return  $m$   $\triangleright$  no transition triggered

```

B.4 Mapping to HyperClaw v2 (PRIMUS) Components

In v2, the metaprotocol's components map to PRIMUS subsystems as shown in Table 2.

Metaprotocol Component	HyperClaw v1	HyperClaw v2
Attention directives	Typed METTA messages	ECAN importance-weighted routing
Admission gates	Heuristic predicates	SUBREP CDS/PDS certificates
Mode register	Enumerated modes	METAMO motive geometry
Geodesic scoring	$f \cdot g$ heuristic	Schrödinger-bridge control
Module routing	Empirical rating table	TRANSWEAVE transfer maps
Context frame	Structured METTA record	ATOMSPACE metagraph region
Fast loop	LLM-centric orchestration	Goal-directed loop with PRIMUS methods
Slow loop	Periodic evaluation + human	Ambient loop + METAMO
Creative intervention	Human approval gate	WILLIAM mining + concept blending

Table 2: Component mapping from HYPERCLAW v1 to v2.

C Quant Finance Workflow: Detailed Architecture

C.1 Module Space Configuration

Space	Backend	Strengths	Capabilities
Math/Writing LLM	Self-hosted or API	Derivations, exposition	Read: full frame. Write: derivations, papers
Code/Creative LLM	Self-hosted or API	Problem-solving, code	Read: full frame. Write: code, suggestions
Critic LLM	Self-hosted or API	Critical review	Read: full frame. Write: critiques
Python Space	Local process	Code execution	Read: workspace. Write: results, artifacts
Data Space	File system	Persistent storage	Read/Write: datasets, results
LaTeX Space	Local compiler	Document production	Read: source. Write: PDF artifacts

Table 3: Module Spaces for the quant finance workflow. Each LLM Space is backend-agnostic; self-hosted open-source models (e.g., fine-tuned LLaMA/Mixtral via vLLM or TGI) are preferred where they match or exceed commercial API quality for the task.

C.2 Workflow Phases as Modes

The slow loop manages transitions through a sequence of strategic modes:

1. **Explore-Methods:** Generate candidate approaches (e.g., fractional-moment volatility, SLT-based classification, compression-based regimes). High autonomy for the fast loop; many parallel experiments. Transition trigger: one method shows promise (metrics exceed baseline by δ).
2. **Refine-Parameters:** Fix the method; optimize hyperparameters, evaluation protocol, and data splits. Moderate autonomy; systematic parameter sweeps. Transition trigger: performance plateaus or budget exhaustion.
3. **Validate:** Test on held-out data with proper transaction-cost modeling. Low autonomy; strict evaluation protocol from the certified method record. Transition trigger: validation passes or fails definitively.
4. **Wrap-Up:** Produce clean deliverables (library, paper, datasets). Build process references certified method CIDs. Transition trigger: all deliverables pass review.

Mode transitions between Explore-Methods and Refine-Parameters (and back, if refinement reveals the method is flawed) may occur multiple times. The pivot from “volatility-based” to

“compression-based” regimes—the key creative intervention in the original workflow—would appear as a return from Refine-Parameters to Explore-Methods, triggered by poor validation results and requiring human approval in v1, or proposed by WILLIAM pattern mining in HYPERCLAW v2.

C.3 The Generate/Critique/Revise Pattern

Algorithm 3 Generate/Critique/Revise cycle

Require: Task description t , context slice c , generator Space G , critic Spaces C_1, C_2 , reviser Space R

Ensure: Revised artifact a' , updated frame entries

```

1:  $a \leftarrow \text{INVOKE}(G, c, \text{GENERATE}, t)$  ▷ e.g., math LLM writes derivation
2:  $\text{critiques} \leftarrow \emptyset$ 
3: for all  $C_i \in \{C_1, C_2\}$  do ▷ fan out in parallel
4:    $\text{critiques} \leftarrow \text{critiques} \cup \{\text{INVOKE}(C_i, (c, a), \text{CRITIQUE})\}$ 
5:  $a' \leftarrow \text{INVOKE}(R, (c, a, \text{critiques}), \text{REVISE})$ 
6: if  $\text{ADMISSIONGATE}(a', \mathcal{F})$  then
7:    $\text{UPDATEFRAME}(\mathcal{F}, \text{artifact} \leftarrow a')$ 
8: else
9:    $\text{ESCALATETO SLOWLOOP}(\text{"revision rejected"}, a')$ 

```

C.4 Preventing the “Forgotten Method” Failure

The catastrophic failure mode—where an LLM substitutes a hallucinated method when building a final library—is prevented by architectural constraint. The certified method record in the context frame is a CID-addressed object containing: (i) the method specification (regime definitions, feature formulas, evaluation protocol), (ii) the specific parameter values, (iii) the experimental evidence (result CIDs), and (iv) a hash chain linking each component to the experimental iteration that validated it.

When the Wrap-Up mode directs an LLM to “build a clean library implementing the working method,” the attention directive’s context slice includes the full certified method specification—not a summary, not a request for the LLM to remember, but the actual typed record. The admission gate for the library output verifies structural consistency with the certified method CID.

In HYPERCLAW v2, this becomes a SUBREP certificate: the library code is an option that must pass CDS admission against the certified method’s motive cone before it can be released as a deliverable.

D Humanoid Robotics Workflow: Detailed Architecture

D.1 Real-Time Attention Budget

The robotics use-case imposes hard timing constraints that the quant-finance case does not. The attention-metaprotocol allocates a per-cycle budget (target: < 500ms) across competing subsystems.

Subsystem	Budget	Priority	Preemptible?
Safety monitor	50ms	Highest (veto)	No
Perception (vision + audio)	100ms	High	No
Emotional appraisal	50ms	Medium-high	Yes
Dialogue generation	150ms	Medium	Yes
Motor planning	100ms	Medium	Yes
Background reasoning	Remainder	Low	Yes

Table 4: Attention budget allocation for real-time robotics.

The safety monitor runs as a non-preemptible background process with veto power: it can suppress any motor command or dialogue output that violates safety constraints, regardless of what the orchestrator has planned.

D.2 Conflict Resolution

When perception modules produce contradictory signals (e.g., visual engagement vs. vocal confusion), the metaprotocol applies weighted evidence fusion:

Algorithm 4 Perceptual conflict resolution

Require: Perception signals $\{(s_i, w_i, src_i)\}$, current scene model $\mathcal{M}$

Ensure: Updated scene model $\mathcal{M}'$

- 1: **for all** conflicting signal pairs (s_j, s_k) where $s_j \perp s_k$ **do**
 - 2: $conf_j \leftarrow w_j \cdot \text{RECENTACCURACY}(src_j)$
 - 3: $conf_k \leftarrow w_k \cdot \text{RECENTACCURACY}(src_k)$
 - 4: **if** $|conf_j - conf_k| > \theta_{clear}$ **then**
 - 5: Accept the higher-confidence signal; log the discrepancy
 - 6: **else**
 - 7: Maintain both hypotheses with respective weights
 - 8: Emit attention directive to gather disambiguating evidence
 - 9: $\mathcal{M}' \leftarrow \text{FUSEINTOSCENEMODEL}(\text{accepted signals}, \mathcal{M})$
-

In HYPERCLAW v2, this becomes PLN evidence fusion with proper truth-value propagation through quantale-annotated factor graphs, and the disambiguating-evidence directive becomes an ECAN-guided attention allocation to the relevant perception channels.

D.3 Pedagogical Mode Transitions

For a classroom tutoring scenario, the slow loop manages transitions through pedagogical modes governed by student-model updates:

1. **Explain:** Present material. Transition: student signals understanding (nod, correct response) $\rightarrow$ Quiz; student signals confusion (frown, hesitation) $\rightarrow$ Re-Explain with different approach.
2. **Quiz:** Test understanding. Transition: correct $\rightarrow$ Advance; incorrect with effort $\rightarrow$ Encourage then Re-Explain; incorrect with distress $\rightarrow$ De-Escalate.
3. **Encourage:** Provide positive reinforcement. Transition: emotional state improves $\rightarrow$ return to previous mode; emotional state worsens $\rightarrow$ De-Escalate.

4. **De-Escalate:** Reduce cognitive load, offer break, redirect. METAMO throttles novelty and reduces engagement pressure. Transition: student re-engages → Explain (gentler pace).

E ASI:Chain Deployment Architecture

E.1 Deployment Topology

A HYPERCLAW instance on ASI:CHAIN consists of three layers:

1. **On-chain layer.** Stores: orchestration-rule CIDs, certified-method CIDs, module-capability contracts, audit summaries (hashes of frame snapshots and transition events), governance hooks (DAO vote thresholds for rule upgrades). Provides: immutable provenance, multi-party trust anchoring, economic settlement for metered resources.
2. **Off-chain MettaCycle layer.** Runs: the actual METTA-IL orchestration logic on MORK-backed ATOMSPACES within Rholang/RSpace executors. Each module Space runs as a capability-scoped Rholang process. Frame state lives in MORK with DAS (Distributed ATOMSPACE) replication across nodes. Proof-of-execution summaries are posted to chain.
3. **Edge layer.** For robotics: on-device executors handle latency-critical perception and motor control, with only strategic decisions and learning updates propagating to the off-chain layer. For quant finance: local Python executors and data stores, with orchestration and LLM calls managed by the off-chain layer.

E.2 Capability Contracts for Module Spaces

Each module Space operates under a capability contract specifying:

Capability	Specification
<code>frame_read</code>	Set of frame field paths this Space may read
<code>frame_write</code>	Set of frame field paths this Space may write
<code>compute_budget</code>	Maximum compute units (tokens, CPU-seconds, GPU-seconds)
<code>data_access</code>	Enumerated datasets with read/write permissions
<code>network_access</code>	Allowed external endpoints (LLM APIs, data feeds)
<code>persistence</code>	Whether the Space may persist state across invocations
<code>audit_level</code>	Granularity of execution logging required

Table 5: Capability contract fields for module Spaces.

Contracts are enforced by the Rholang runtime—a Space literally cannot access frame fields outside its `frame_read` set, because the capability tokens it holds do not include those paths.

E.3 Lifecycle Management

Algorithm 5 Shadow/elevate/rollback deployment

Require: New configuration C_{new} , current configuration C_{cur}

Ensure: Deployed configuration C_{active}

```

1:  $cid_{cur} \leftarrow \text{CHECKPOINT}(C_{cur})$  ▷ save rollback point
2:  $\text{DEPLOYSHADOW}(C_{new})$  ▷ run alongside  $C_{cur}$ , output discarded
3: for  $t \leftarrow 1$  to  $T_{shadow}$  do
4:    $metrics \leftarrow \text{COMPAREOUTPUTS}(C_{cur}, C_{new})$ 
5:   if  $metrics.divergence > \epsilon_{abort}$  then
6:      $\text{ABORT}(C_{new}); \text{return } C_{cur}$ 
7:  $\text{ELEVATE TODUALRUN}(C_{new})$  ▷ both produce output; compare
8: for  $t \leftarrow 1$  to  $T_{dual}$  do
9:    $kpis \leftarrow \text{EVALUATEKPIs}(C_{new})$ 
10:  if  $kpis$  stable within tolerance bands then
11:     $\text{ELEVATE}(C_{new}); \text{return } C_{new}$ 
12:  $\text{ROLLBACK}(cid_{cur}); \text{return } C_{cur}$  ▷ KPIs did not stabilize

```

F HyperClaw v1 $\rightarrow$ v2: A Concrete Upgrade Path

The following table traces the upgrade path for the quant-finance use-case, showing how each HYPERCLAW v1 component is progressively augmented or replaced by PRIMUS cognitive processes in v2.

Function	HyperClaw v1	HyperClaw v2
Mathematical derivation	LLMs generate; peer LLMs critique	PLN derives with uncertainty; LLMs assist exposition
Code generation	LLM generates Python; executor runs	MOSES/GEO-EVO evolves programs; LLMs scaffold
Result evaluation	LLM analyzes; heuristic checks	PLN + factor graphs propagate evidence
Creative pivots	Human proposes; system scores $f \cdot g$	WILLIAM mines patterns; concept blending proposes; human approves major shifts
Method certification	CID-linked experiment records	SUBREP CDS/PDS certificates with formal guarantees
Multi-LLM routing	Empirical quality ratings	TRANSWEAVE transfer maps with commutator bounds
Attention allocation	Metaprotocol rules	ECAN with fluid-dynamic transport on MORK
Goal management	Mode register + human steering	METAMO motive geometries with contractive updates
Background insight	None (human intuition)	Ambient loop: pattern mining, concept blending, PLN consolidation

Table 6: Upgrade path from HYPERCLAW v1 to v2.

Each upgrade is itself managed by the shadow/elevate/rollback lifecycle (Algorithm 5), so the transition from v1 to v2 is incremental and reversible—individual components can be upgraded independently, and the system can fall back to LLM-based processing for any component where the PRIMUS replacement is not yet mature. HYPERCLAW v2 is not a different system; it is v1 with deeper cognitive machinery behind the same orchestration interface.